DPS912 Lab4: I/O Control

Due: Wednesday, June 3, 2026 (11:59pm)

In this lab, you will step into the role of a **Linux device driver developer**. Instead of writing ordinary user programs, you will work inside the kernel and control a simulated hardware device from user space using standard Linux I/O mechanisms.

Modern operating systems do not allow applications to touch hardware directly. Instead, applications must go through **device drivers**, which act as trusted intermediaries between user programs and the kernel. Your task in this lab is to complete the missing pieces of such a driver so that:

- A user-space program can **read data from a kernel buffer**, and
- The user program can **control the device's behavior** using ioctl() commands.

What this "hardware device" does

The driver simulates a simple hardware device:

- It owns a **kernel buffer**.
- A kernel thread updates this buffer **once per second**.
- Your job is to make this buffer visible to user space using a **read()** function.
- You will also add control logic using **ioctl()** so a user program can:
    - Pause the updates (HARDWARE_DEVICE_HALT)
    - Resume the updates (HARDWARE_DEVICE_RESUME)

This mimics how real drivers allow applications to both **read data** and **send control commands** to devices like sensors, cameras, or network cards.

What you will implement

In this lab, your task is to complete the missing sections of a hardware device driver. Most of the code is already provided, and you are required to implement the *read* and *ioctl* functions. Refer to the **TODO** comments in the driver code for guidance.

- The source code for the hardware device driver can be found at Makefile, hardwareDevice.h, and hardwareDevice.c.
  Scripts to load the module into the kernal and unload the module from the kernel can be found at load.sh and unload.sh.
  A script that unloads, builds and loads the kernel module all in one can be found at build.sh. You may need to modify the permissions of these scripts using the command: **$ chmod 777 *.sh**

- The hardware device driver operates as a buffer managed by a thread that updates its content once per second. Your responsibility is to implement the ***read*** function, allowing a user-space program to access the buffer's contents. Additionally, you need to implement the ***ioctl*** function to support two commands: **HARDWARE_DEVICE_HALT** and **HARDWARE_DEVICE_RESUME**.
    - The **HARDWARE_DEVICE_HALT** command stops the buffer updates.
    - The **HARDWARE_DEVICE_RESUME** command restarts the periodic updates.
    - Details for **HARDWARE_DEVICE_HALT** and **HARDWARE_DEVICE_RESUME** can be found in the header file **hardwareDevice.h**.
- To test your implementation, a user-space program is provided in Makefile and userHardware.cpp.

- This program performs the following sequence:
    - Reads the buffer from kernel space three times at three-second intervals.
    - Issues the **HARDWARE_DEVICE_HALT** command.
    - Reads the buffer three more times, again at three-second intervals.
    - Issues the **HARDWARE_DEVICE_RESUME** command.
    - Reads the buffer three final times at three-second intervals before completing.
- Take note of the directory structure. The Makefile for **userHardware.cpp** requires access to the kernel header file, **hardwareDevice.h**, which it expects to be in a directory with the relative path **../kernel**.

Research Questions

1) How are ioctl command numbers created using _IO, _IOR, _IOW, _IOWR macros? Why do we need to encode direction (read/write) and size into the command number?
2) Compare the output of userHardware.cpp and the kernel dmesg log. Do they always match exactly? Why or why not?

Assignment Submission:

- Complete all steps, Add all output-screenshot and explanations (if required) to a MS-Word file.

- Add the following declaration at the top of MSWORD file
/***
***
* DPS912-Lab4
* I declare that this lab is my own work in accordance with Seneca Academic Policy.
* No part of this assignment has been copied manually or electronically from any other source
* (including web sites) or distributed to other students.
*
* Name: ____________________ Student ID: ________________ Date: _____________________
*
*

****
**/

- Please submit the Source code (zip all .c, .h, and makeFiles)
- Please answer the following two declarations:
    - D1) On a scale from 1 to 5, **How much did you use generative AI to complete this assignment?**
        - where:
        - **1** means you did not use generative AI at all
        - **2** means you used it very minimally
        - **3** means you used it moderately
        - **4** means you used it significantly
        - **5** means you relied on it almost entirely
        - **Your answer :**
    - D2) On a scale from 1 to 5, **How confident are you in your understanding of the generative AI support you utilized in this assignment, and in your ability to explain it if questioned?**
        - where:
        - **1** means "Not confident at all – I do not understand the generative AI support I used and cannot explain it."
        - **2** means "Slightly confident – I understand a little, but I have many uncertainties."
        - **3** means "Moderately confident – I understand the majority of the support, though some parts are unclear."
        - **4** means "Very confident – I understand most of the AI support well and can explain it with minor gaps."
        - **5** means "Extremely confident – I fully understand the generative AI support I used and can clearly explain or justify it if asked."
        - **Your answer :**

Important Note:

- **LATE SUBMISSIONS for labs**. Late lab submissions will receive a **10% deduction per day**. Labs submitted more than **three days late will receive a grade of zero (0)**.
    - If you are unable to finish the lab on time, **submit whatever you have before the deadline** to avoid an automatic zero.
- This labs should be submitted along with a video-recording which contains a detailed walkthrough of solution. Without recording, the assignment can get a maximum of 1/3 of the total.
    - Note: In case you are running out of time to record the video, you can submit the assignment (source code + screenshots) by the deadline and submit the video within 24 hours after the deadline.